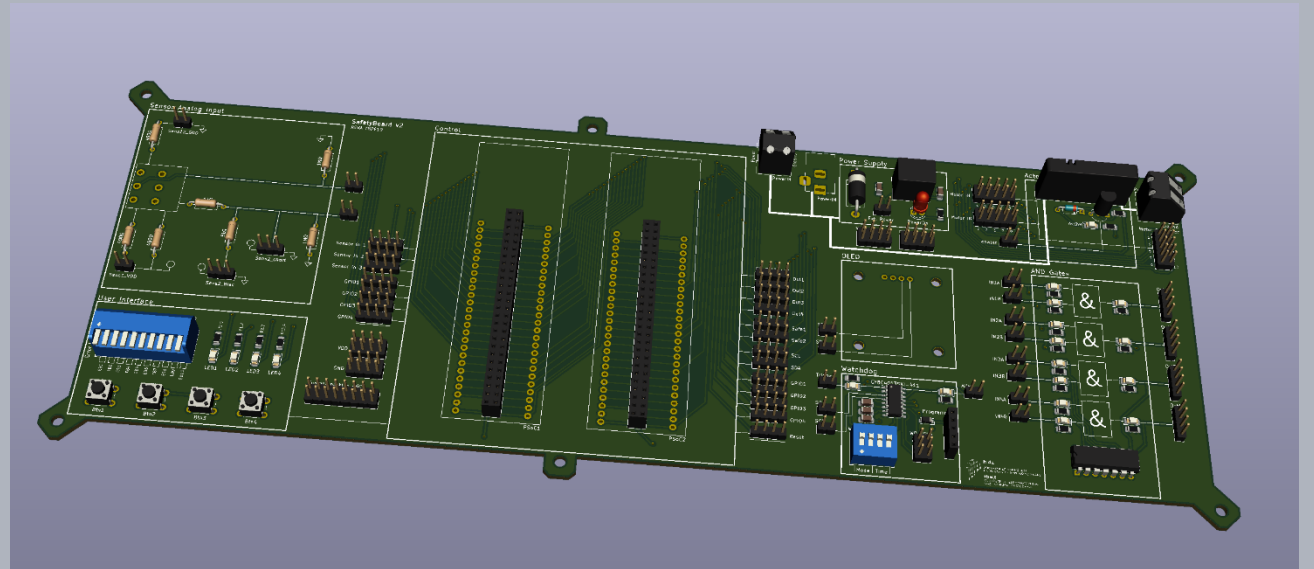


Safety in Embedded Control Systems

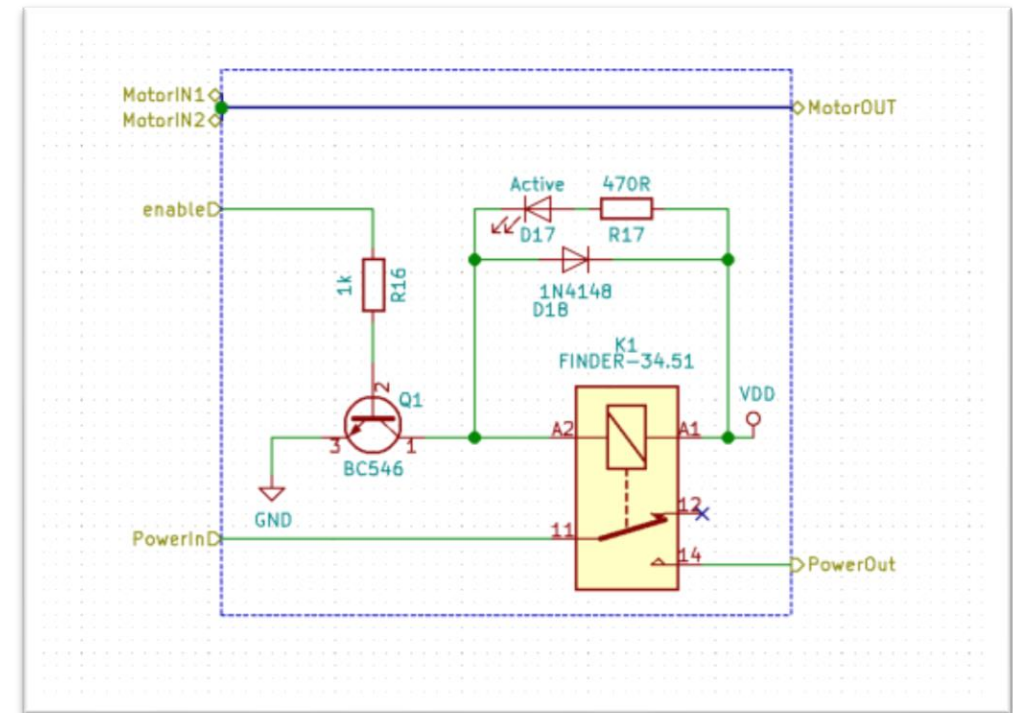
Actor and System monitoring

Prof. Dr.-Ing. Peter Fromm



Content

- Reliability Status of our System
- Output Supervision
- System Level Supervision
 - Internal Watchdog
 - External Watchdog
- Moving towards a redundant controller architecture



Qualitative System Verification

Id	System Requirements	ASIL _{req}	HW / SW Requirement	Status
SR1	The brake signal must be reliable	C	- SW: Check brake pedal operation power on - SW: Ensure reliable reading of "pressed state" - HW: Provide 1oo2 brake signal - HW: Provide diagnostics to detect short circuits / broken lines of brake pedal	   
SR2	The gas pedal signal must be reliable	C	- HW: Add resistors to detect broken / short circuit connections of gas pedal - HW: Provide 1oo2 pedal signal - HW: Use different technologies to read both channels - HW: Use inverted signals - SW: Drift of potentiometer must be detected - SW: Short circuits / broken lines of gas pedal must be detected by complex device driver	     
SR3	The car may only start after an explicit ignition sequence	QM	- SW: A clear press of the ignition button must be detected - SW: The engine may only start if the gas pedal is depressed	 
SR4	The control speed value must be correctly calculated	C	- SW: Brake signal must override gas pedal - SW: Wrong calculations of control speed value must be detected - SW: Supervise / Limit PID Control Parameters - SW: Driving direction may only be changed if engine is stopped - SW: Reliable monitor communication link - SW: Controlspeed will be calculated with different algo's on both controllers - SW: State Machine protection	      

Qualitative System Verification

Id	System Requirements	ASIL _{req}	HW / SW Requirement	Status
SR5	The engine currentspeed must be in line with the controlspeed value	C	- HW: Engine power must be deactivated in case of critical failure - HW: Engine power must be controlled by independent hardware - HW: Engine current must be supervised and limited - SW: Engine Blocking must be detected - SW: Current speed must be compared with target speed - SW: Limit reverse speed - SW: Engine must be explicitly activated	
SR6	The system status must be shown to the driver	A	- SW: Provide status / error information to driver - SW: Detect freeze of OLED display	
SR7	The system integrity must be supervised	C	- HW: Provide an external watchdog, which turns of the actuator in case of a critical error - HW: Power supply must be supervised, bursts must be avoided - HW: The integrity of the controller (MCU) must be supervised - SW: The integrity of the OS must be supervised - SW: ISR bursts must be detected - SW: Low Power must be detected	

Where we are: System Level Reliability

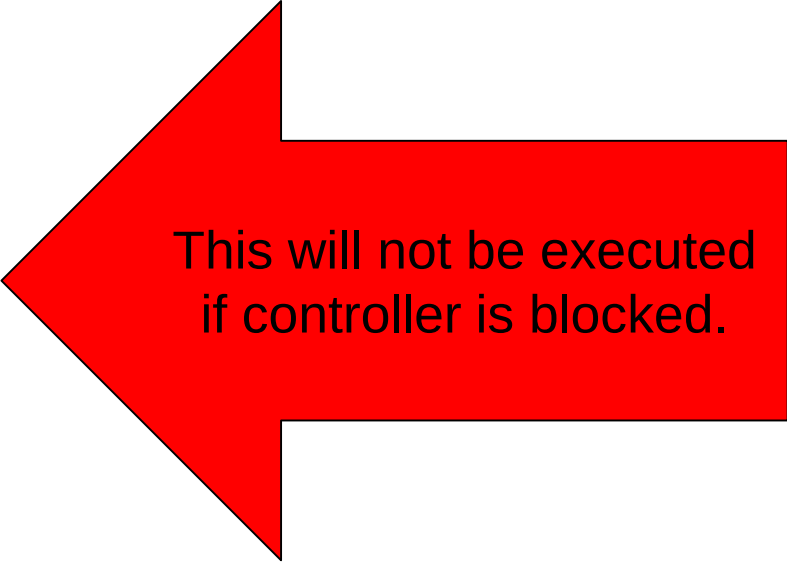
- Until now, we have only introduced error handling by software
- This only works, if the controller itself is in an operational state!

```
/**
 * Central error handler
 * At the moment only display of the error
 */
void ERROR_centralHandler(CAR_error_t error)
{
    DRIVINGPARAM_data_t param = DRIVINGPARAM_INIT_DATA;

    param.m_maxSpeed = 0;
    param.m_direction = DIRECTION_NEUTRAL;
    param.m_error = error;

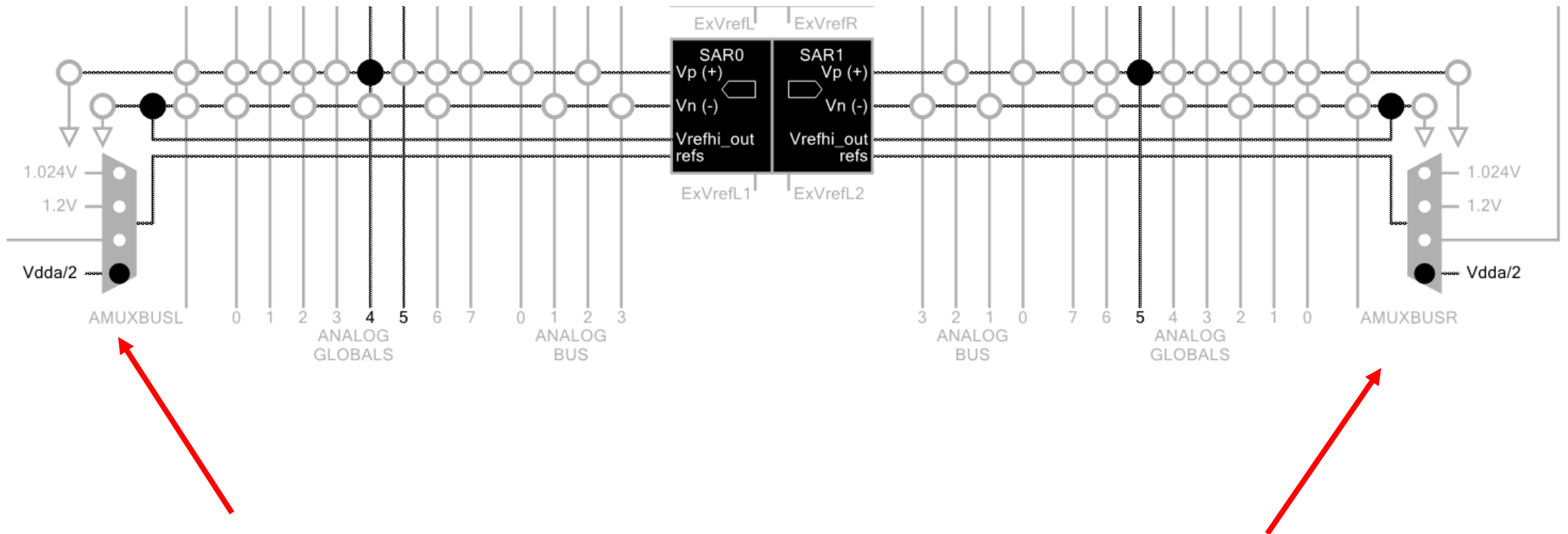
    ENG_ResetPID(ENG_ALL);

    RTE_DRIVINGPARAM_set(&SO_DRIVINGPARAM_signal, param);
    RTE_DRIVINGPARAM_pushPort(&SO_DRIVINGPARAM_signal);
}
```

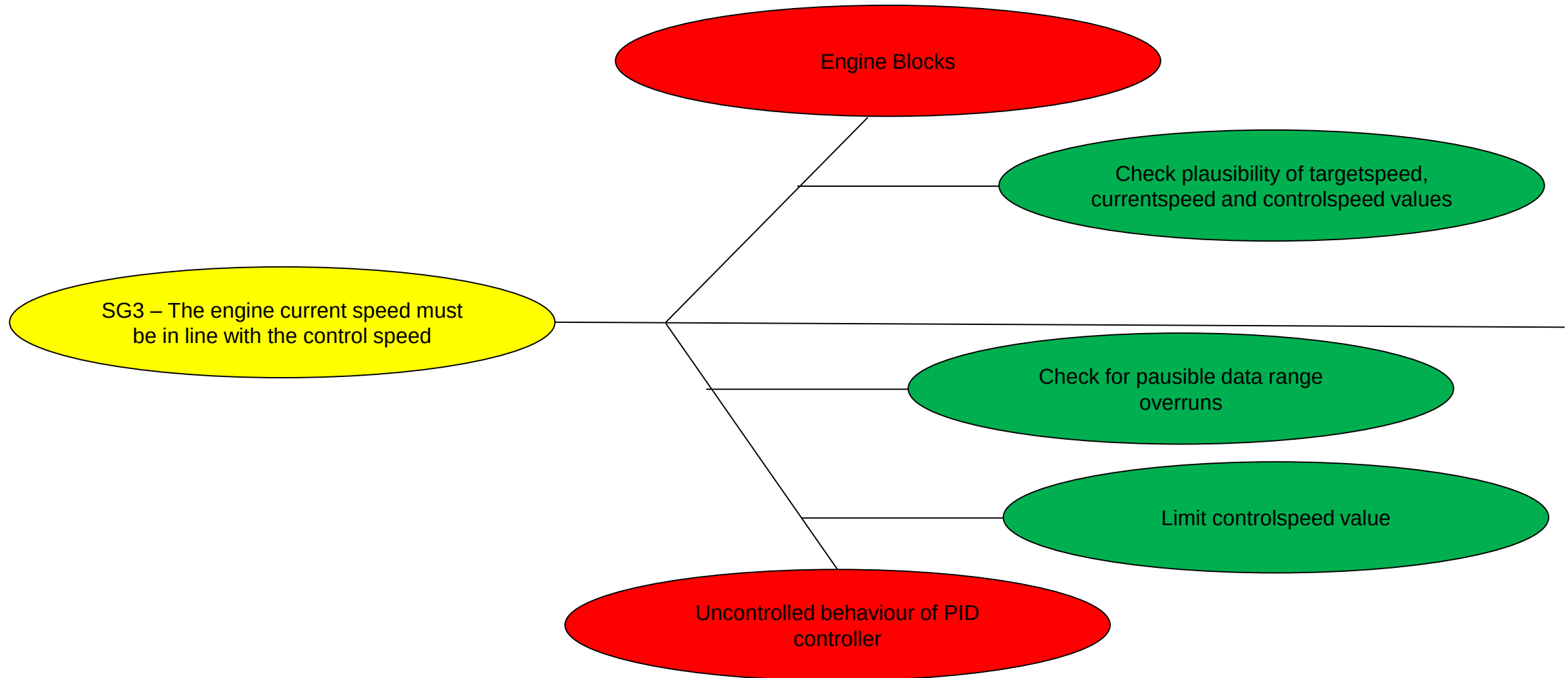


This will not be executed
if controller is blocked.

Common Cause Failure: ADC Power Supply

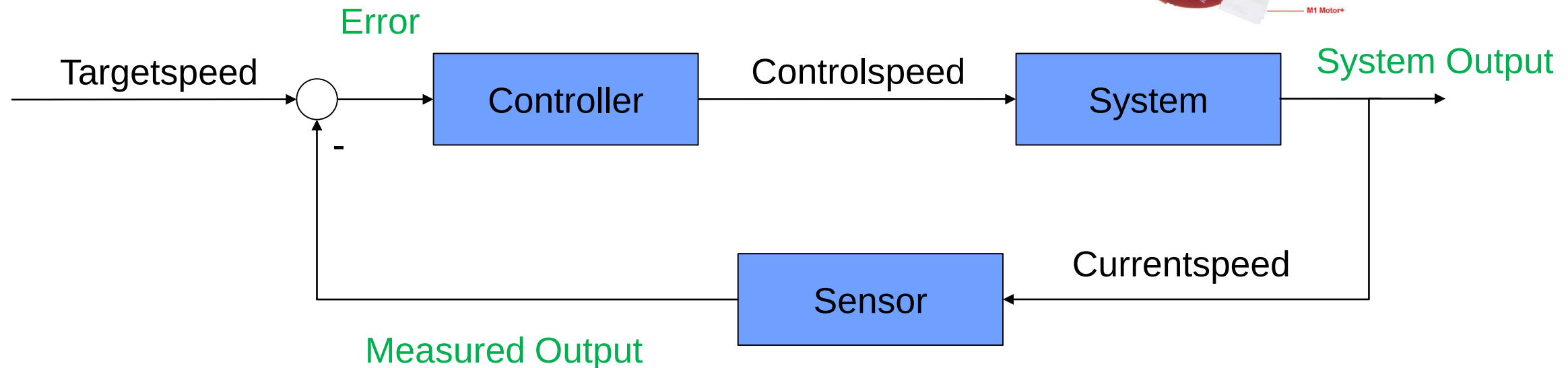


Let's start with the actor supervision

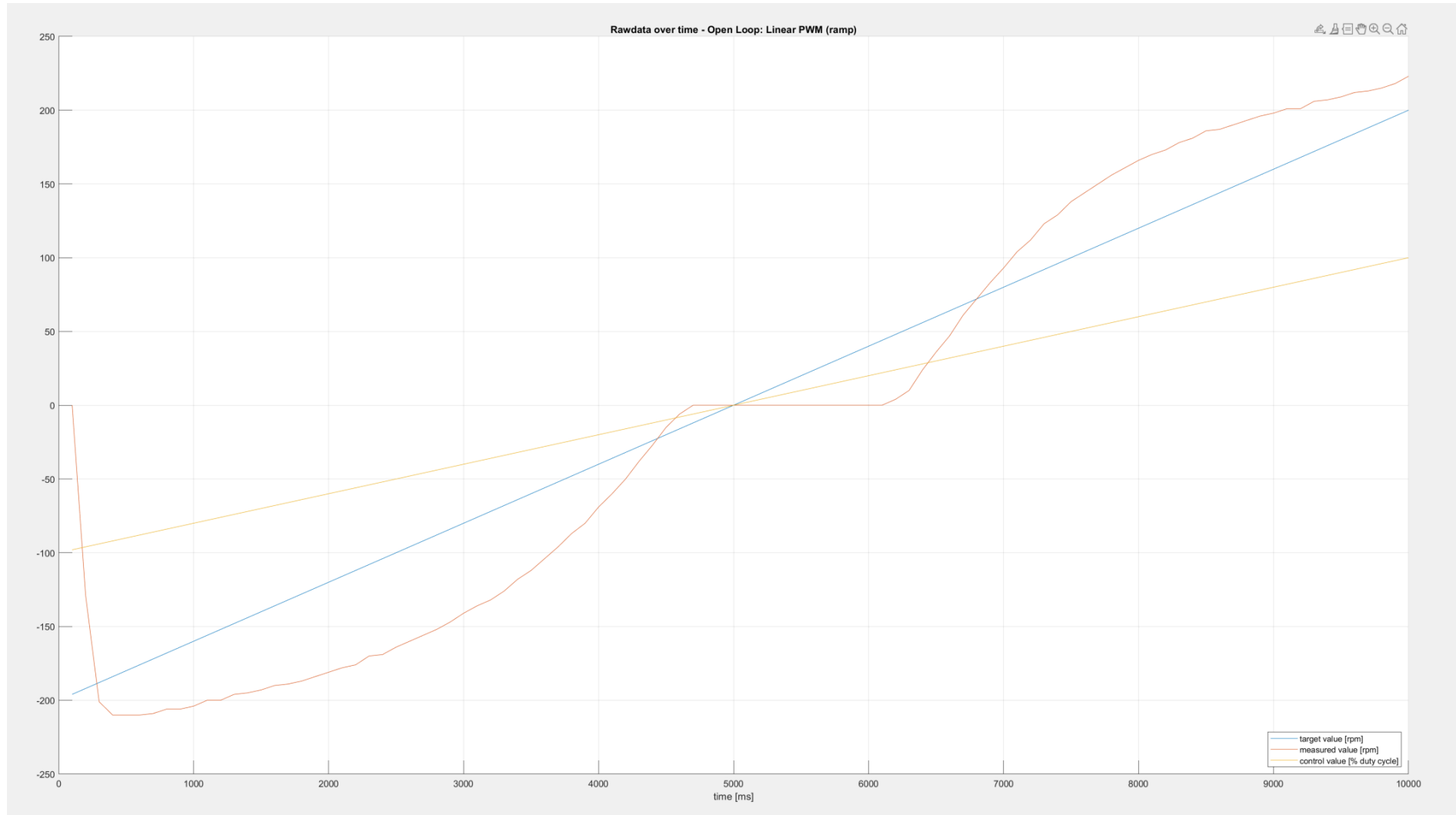


Plausibility of speed values

- Targetspeed – Speed requested by the user
- Controlspeed – Speed value given to the engine driver (H-bridge)
- Currentspeed – Speed measured at the engine

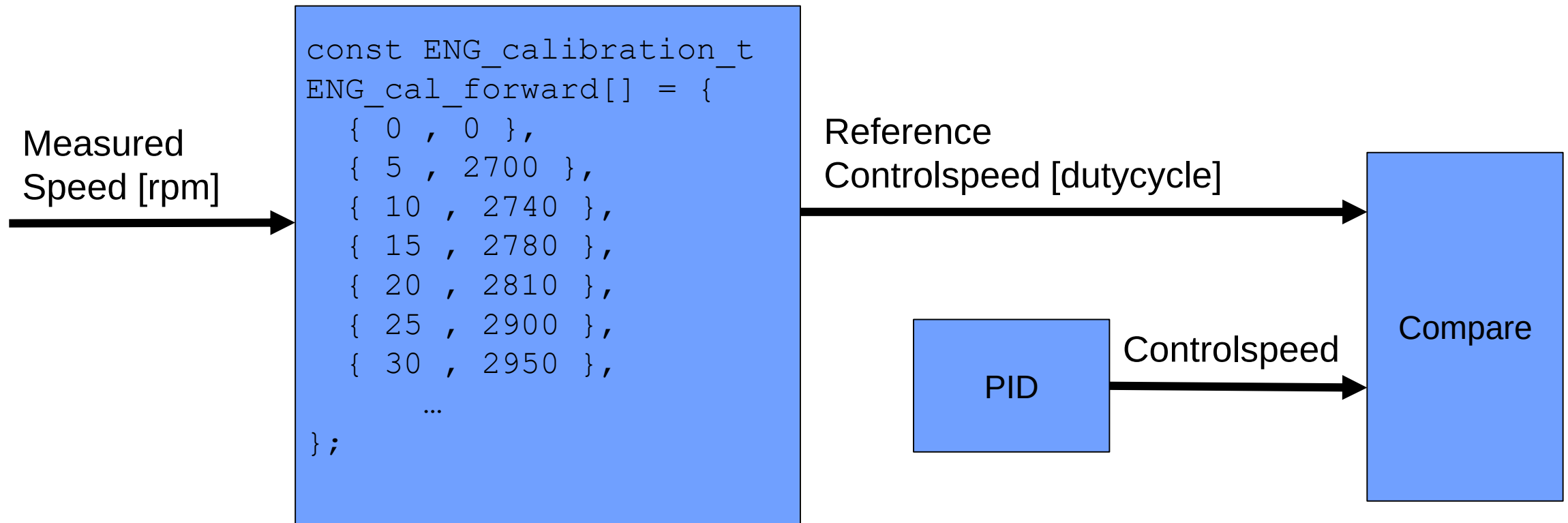


Plausibility of speed values – open loop behaviour



General idea

- We compare the calculated controlvalue of the PID controller with the expected controlvalue of the open loop calibration curve.
- These values of course will not be identical, but should be in the same range.
- If the calculated controlvalue is much higher than the reference value, the engine is blocking.

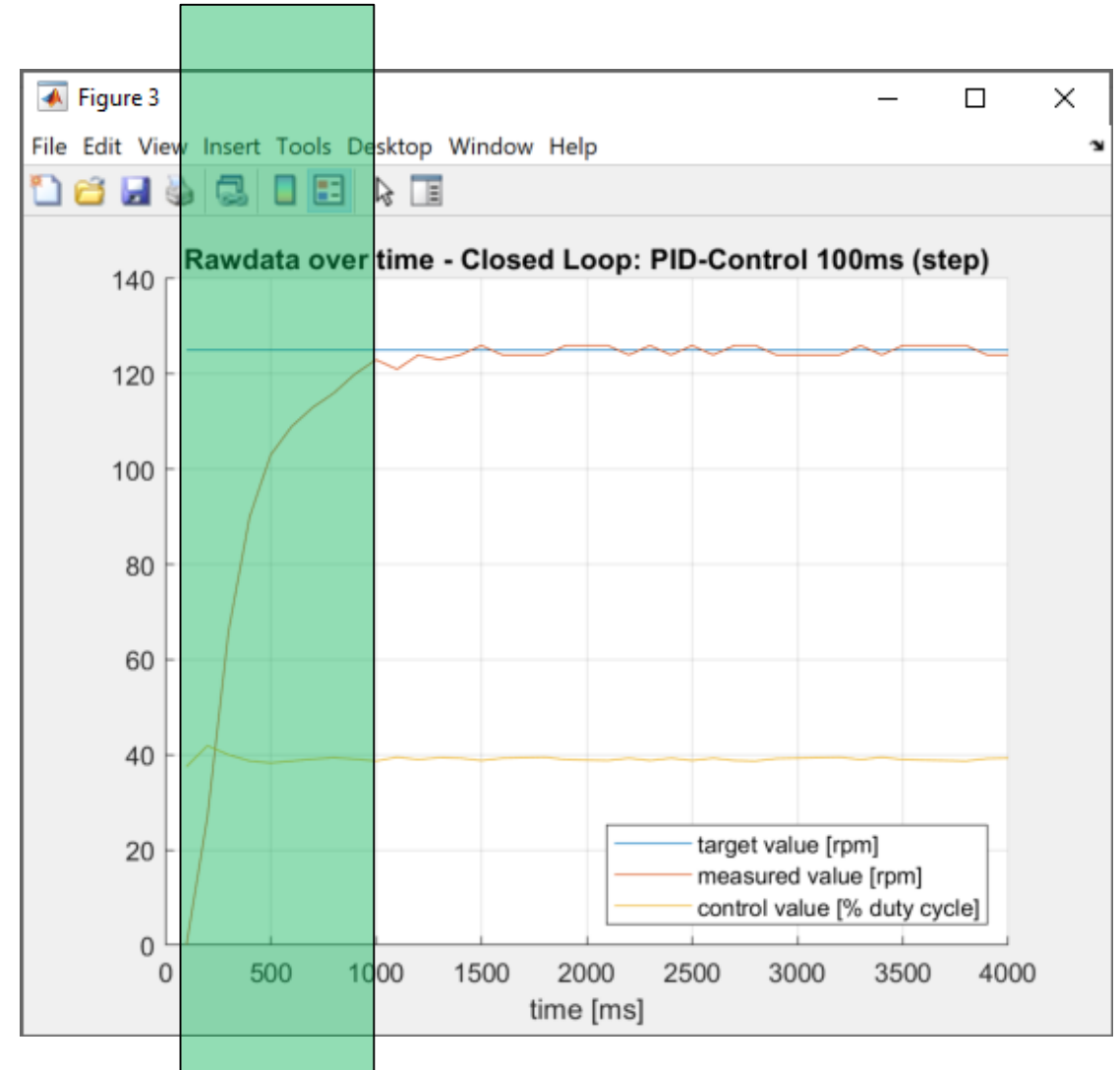


Challenge: Dynamics / Time behaviour

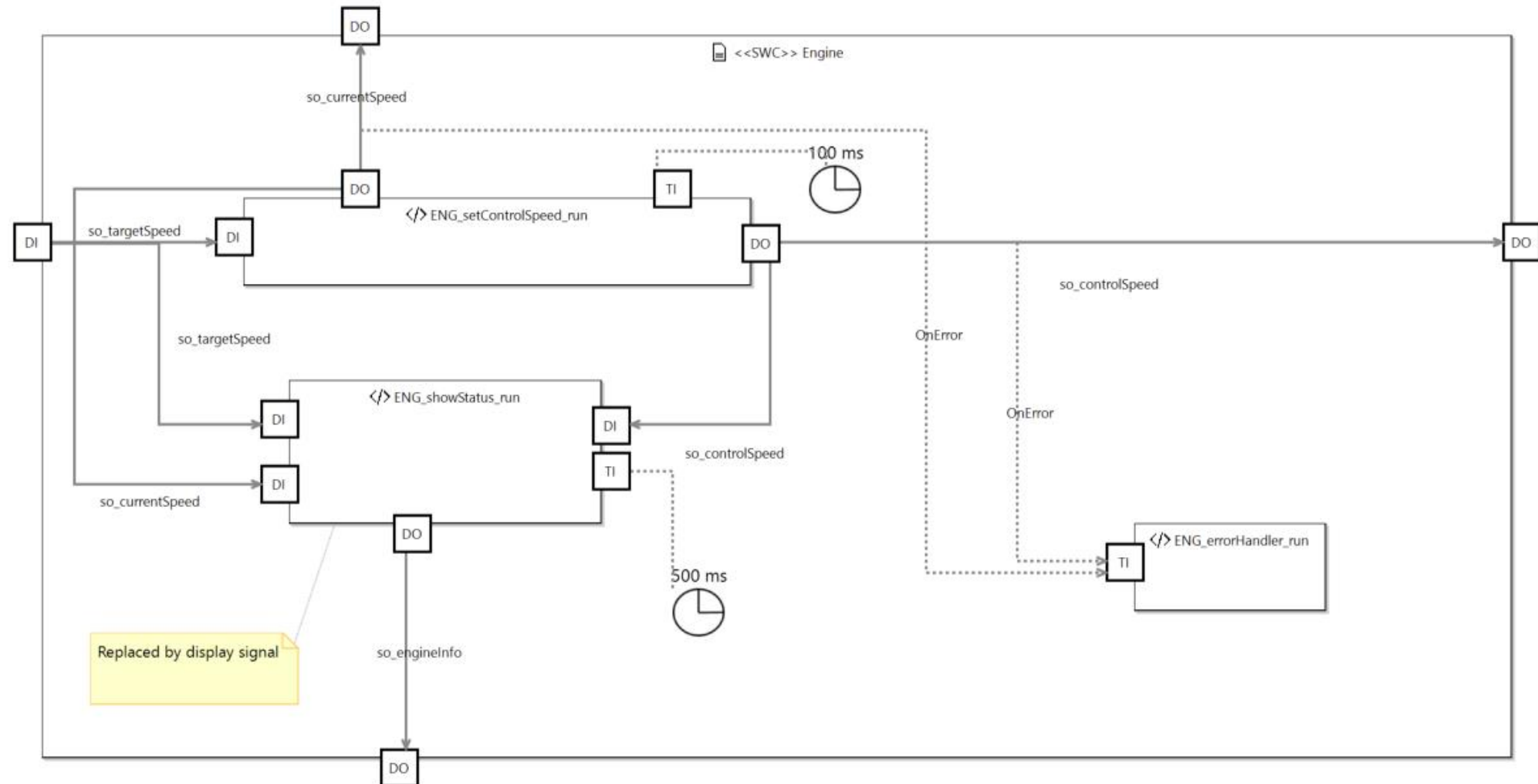
General logic:

- If the calculated control speed is much higher than the control according to the open loop calibration curve, we consider the engine to be blocking
- Problem: This logic would also stop the engine in case of a fast acceleration

We need kind of a low pass filter, e.g. error counter.



General design of the output side



Resulting code of the SWC_Engine runnable - 1

```
void ENGINE_setControlSpeed_run()
{
    RC_t result = RC_SUCCESS;

    //Get the ticktime of this runnable
    uint16_t ticktime = RTE_getTickTime(RTE_cyclicActivationTable, RTE_cyclicActivation_size, ENGINE_setControlSpeed_run);

    //Get the targetspeed
    SPEED_data_t targetSpeed = SPEED_INIT_DATA;
    targetSpeed = RTE_SPEED_get(&SO_TARGETSPEED_signal);

    DRIVINGPARAM_data_t allowedTargetSpeed = RTE_DRIVINGPARAM_get(&SO_DRIVINGPARAM_signal);

    LOG_I("ENGINE_setControlSpeed_run", "Max allowed: %d", allowedTargetSpeed.m_maxSpeed);

    //Limit speed if required
    if (abs(targetSpeed.m_rpm) > allowedTargetSpeed.m_maxSpeed)
    {
        if (targetSpeed.m_rpm < 0)
        {
            targetSpeed.m_rpm = - allowedTargetSpeed.m_maxSpeed;
        }
        else
        {
            targetSpeed.m_rpm = allowedTargetSpeed.m_maxSpeed;
        }
    }
}
```

Software clipper, no
escalation to error handler.

Resulting code of the SWC_Engine runnable - 2

```
//Calculate the controlspeed
PWM_data_t controlSpeed = PWM_INIT_DATA;

//And get the measured speed
result = RTE_SPEED_pullPort(&SO_CURRENTSPEED_signal);
if (RC_SUCCESS != result)
{
    //Signal status will be set invalid and error handler will be called
    return;
}

SPEED_data_t measuredSpeed = RTE_SPEED_get(&SO_CURRENTSPEED_signal);

//Calculate controlvalue and get measured speed
result = ENG_SetEngineClosedLoopPID(ENG_1,ticktime, targetSpeed.m_rpm, measuredSpeed.m_rpm, &controlSpeed.m_dutycycle);
if (RC_SUCCESS != result)
{
    //Signal status will be set invalid and error handler will be called
    RTE_PWM_setStatus(&SO_CONTROLSPEED_signal, RTE_SIGNALSTATUS_INVALID);
    return;
}
```

In case of a driver error,
direct escalation to central
error handler

In case of PID algorithm
error, direct escalation to
central error handler

Resulting code of the SWC_Engine runnable - 3

```
//Let's check the plausibility of the values
result = ENG_checkControlspeedPlausibility(measuredSpeed.m_rpm, controlSpeed.m_dutycycle);
if (RC_SUCCESS != result)
{
    //Signal status will be set invalid and error handler will be called
    RTE_PWM_setStatus(&SO_CONTROLSPEED_signal, RTE_SIGNALSTATUS_INVALID);
    //We may not return here, as otherwise the controlspeed has no change to follow a fast targetspeed increase
}
else
{
    //Reset Error counter
    ENGINE__errorCounter = 0;
}

//LOG_I("ENG_setControlSpeed_run", " T: %d, M: %d, C: %d", targetSpeed.m_rpm, measuredSpeed.m_rpm, controlSpeed.m_dutycycle);

//And write controlspeed to the engine
result = RTE_PWM_set(&SO_CONTROLSPEED_signal, controlSpeed);
result = RTE_PWM_pushPort(&SO_CONTROLSPEED_signal);
```

The plausibility check
requires a bit more
tolerance

Checking the control speed plausibility

```
/**
 * Check the plausibility of current speed and control speed
 * \param sint16_t measuredSpeed      : IN - The current measured speed [RPM]
 * \param sint16_t controlSpeed      : IN - The current controlspeed [PWM]
 * \return RC_SUCCESS
 */
RC_t ENG_checkControlspeedPlausibility(sint16_t measuredSpeed, sint16_t controlSpeed)
{
    //Search for the reference controlspeed form the open loop table
    //As we only look for a rough plausibility, we will only check the forward table
    //Values are similar enough for this
    measuredSpeed = abs(measuredSpeed);
    controlSpeed = abs(controlSpeed);
    sint16_t refControlSpeed = 0;
    for (uint16_t i = 0; i < ENG_cal_forward_size-1; i++)
    {
        if (measuredSpeed >= ENG_cal_forward[i].m_targetspeed && measuredSpeed < ENG_cal_forward[i+1].m_targetspeed)
        {
            refControlSpeed = (ENG_cal_forward[i].m_controlspeed + ENG_cal_forward[i+1].m_controlspeed) / 2;
            //Rough approximation
            break;
        }
    }

    LOG_I("ENG_SpeedPlausibility", "C : %d R: %d", controlSpeed, refControlSpeed);

    //We mainly want to detect blocking situation
    if (controlSpeed - refControlSpeed > ENG_PWM_MAXDELTA) return RC_ERROR_INVALID_STATE; else return RC_SUCCESS;
}
```


The output central error handler

```
#define ERRORCOUNT_MAX 5

void ENGINE_errorHandler_run()
{
    if (ENGINE__errorCounter >= ERRORCOUNT_MAX)
    {
        PWM_data_t controlSpeed = PWM_INIT_DATA;
        controlSpeed.m_dutycycle = 0;
        RC_t result = RTE_PWM_set(&SO_CONTROLSPEED_signal, controlSpeed);
        result = RTE_PWM_pushPort(&SO_CONTROLSPEED_signal);

        //Fire event to central state machine
        EVENT_data_t ev = EVENT_INIT_DATA;
        ev.m_ev = EVENT_error;
        result = RTE_EVENT_set(&SO_EVENT_signal, ev);

        ERROR_centralHandler(ERROR_ACTOR);
    }
    else
    {
        ENGINE__errorCounter++;
    }

    LOG_I("ENG_errorHandler_run", "Error engine");
}
```

And don't forget to reset all PID controller states after a restart....

```
/**
 * Reset the internal controller
 * \param ENG_id_t id      : IN - The engine id
 * \return RC_SUCCESS
 */
RC_t ENG_ResetPID(ENG_id_t id)
{
    ENG__integralError = 0;
    ENG__lastError = 0;
}
```

If you forget this, the system will very likely directly enter an error state again after resuming.

Qualification of the PID controller – Code Review

```
RC_t ENG_SetEngineClosedLoopPID(ENG_id_t id, uint16_t ticktime, sint16_t targetspeed, sint16_t measuredspeed, sint16_t* controlspeed)
{
    float32_t error = 0;

    float32_t differentialError = 0;
    RC_t result = RC_ERROR;

    //Useless code to avoid MISRA warning
    id = id;

    //Comfort check: Avoid low speed beeping
    if (abs(targetspeed) < ENG_MINRPM)
    {
        targetspeed = 0;
    }

    //Control logic, all calculations using float32_t
    error = targetspeed - measuredspeed;
    ENG__integralError += (error * ticktime) / 1000;
    differentialError = (error - ENG__lastError) * (1000/ticktime);
    ENG__lastError = error;

    //check I and D (TODO - not used at the moment) error for overrun
    if (ENG_CONTROL_KI * ENG__integralError > ENG_PWM_MAXDUTY) ENG__integralError = ENG_PWM_MAXDUTY/ENG_CONTROL_KI;
    if (ENG_CONTROL_KI * ENG__integralError < -ENG_PWM_MAXDUTY) ENG__integralError = -ENG_PWM_MAXDUTY/ENG_CONTROL_KI;

    float calSpeed = 0;
    calSpeed = (ENG_CONTROL_KP * error + ENG_CONTROL_KI * ENG__integralError + ENG_CONTROL_KD * differentialError);

    //LOG_I("ENG_SetEngineClosedLoopPID","T: %d M: %d C: %d, EP: %d EI: %d", targetspeed, measuredspeed, calSpeed, (sint32_t)error, (sint32_t)integralError);

    //Check for possible overrun
    if (calSpeed > ENG_PWM_MAXDUTY) calSpeed = ENG_PWM_MAXDUTY;
    if (calSpeed < -ENG_PWM_MAXDUTY) calSpeed = -ENG_PWM_MAXDUTY;
    if (abs((sint16_t)calSpeed) < ENG_PWMMIN) calSpeed = 0;

    *controlspeed = (sint16_t)calSpeed;

    return RC_SUCCESS;
}
```

Qualification of the PID controller – Code Review

The code is not too bad

- Some minor MISRA warnings about missing brackets
- Special care is take to avoid data type overruns
- Values are explicitly set to valid data range
- Clippers have been added to avoid critical controller values

Design might be a bit more object oriented, though...

Qualitative System Verification

Id	System Requirements	ASIL _{req}	HW / SW Requirement	Status
SR5	The engine currentspeed must be in line with the controlspeed value	C	- HW: Engine power must be deactivated in case of critical failure - HW: Engine power must be controlled by independent hardware - HW: Engine current must be supervised and limited - SW: Engine Blocking must be detected - SW: Current speed must be compared with target speed - SW: Limit reverse speed - SW: Engine must be explicitly activated	      
SR6	The system status must be shown to the driver	A	- SW: Provide status / error information to driver - SW: Detect freeze of OLED display	 
SR7	The system integrity must be supervised	C	- HW: Provide an external watchdog, which turns of the actuator in case of a critical error - HW: Power supply must be supervised, bursts must be avoided - HW: The integrity of the controller (MCU) must be supervised - SW: The integrity of the OS must be supervised - SW: ISR bursts must be detected - SW: Low Power must be detected	     

Now let's have a look what happens if the system itself crashes

- Malconfiguration of the OS
- Crash of the OS
- Oscillating interrupt
- ...



Some bad testcases

Some bad user
code!!!

```
void HMI_injectError_run()
{
    //Read DIP register
    DIP_data_t dip = DIP_INIT_DATA;
    RTE_DIP_pullPort(&SO_DIP_signal);
    dip = RTE_DIP_get(&SO_DIP_signal);

    //LOG_I("HMI_injectError_run","DIP = %x", dip.byte);

    if (1 == dip.bit.d0)
    {
        //Error Injection shutdown
        LOG_E("HMI_injectError_run","Shutting system down");
        ShutdownOS(E_OK);
    }

    if (1 == dip.bit.d1)
    {
        //Error Injection slow down system
        LOG_E("HMI_injectError_run","Corrupted time base");
        CancelAlarm(alm_10ms);
        SetRelAlarm(alm_10ms,100,100);
    }

    if (1 == dip.bit.d2)
    {
        //Error Injection slow down system
        LOG_E("HMI_injectError_run","while(1)");
        while(1){;}
    }
}
```

Erika is not a safe
OS. All properties are
public!



Observed behavior

Testcase 1 – ShutdownOs()

- Engine speed is frozen
- Application code for reading the ADC, calculating the targetspeed and setting the PWM register is not executed anymore

Testcase 2 – Changing the time base

- Engine starts to jump – even more dangerous
- Caused by some bad design faults, e.g. assuming that the configured cycle time also is the physical cycle time

```
/**
 * Returns the ticktime for the given runnable
 * \param RTE_cyclicTable_t const *const table : [IN] table to be processed
 * \param uint16_t tableSize : [IN] table size
 * \return ticktime in ms, 0 in case runnable is not listed
 */
uint16_t RTE_getTickTime(RTE_cyclicTable_t const *const table, uint16_t tableSize, RTE_pRunnable pRun)
{
    for (uint16_t i = 0; i < tableSize; i++)
    {
        if (table[i].run == pRun)
        {
            return table[i].cycleTime;
        }
    }

    //Error, runnable not found
    return 0;
}
```


Watchdog

A watchdog is a component in a system, which needs to be triggered in regular intervals. It mainly serves to detect a “frozen” system, caused e.g. by

- Endless loops
- Wild interrupts
- OS shutdowns

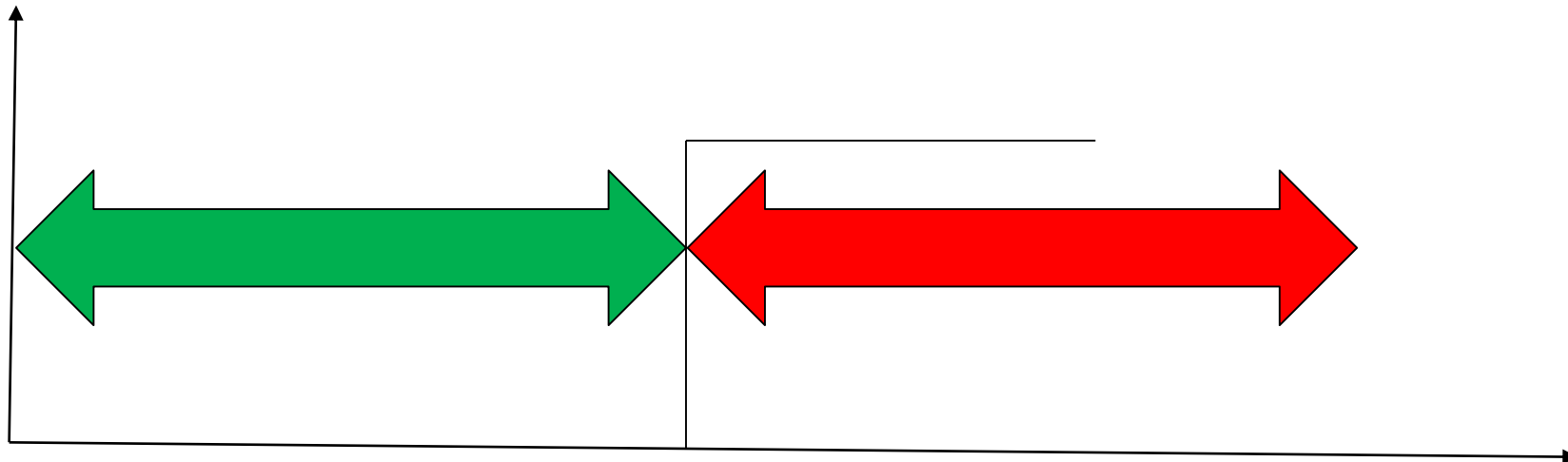
In case the watchdog is not triggered, a hardware action is initiated, often a reset of the microcontroller.

Timeout or Alive Watchdog

- The watchdog needs to be triggered until a certain timeframe (few ms up to few s, depending upon configuration)
- Typically triggered several times within the deadline

- Available in most controllers
- Error handling upon MCU freeze

- Very simple trigger, no real timing synchronisation

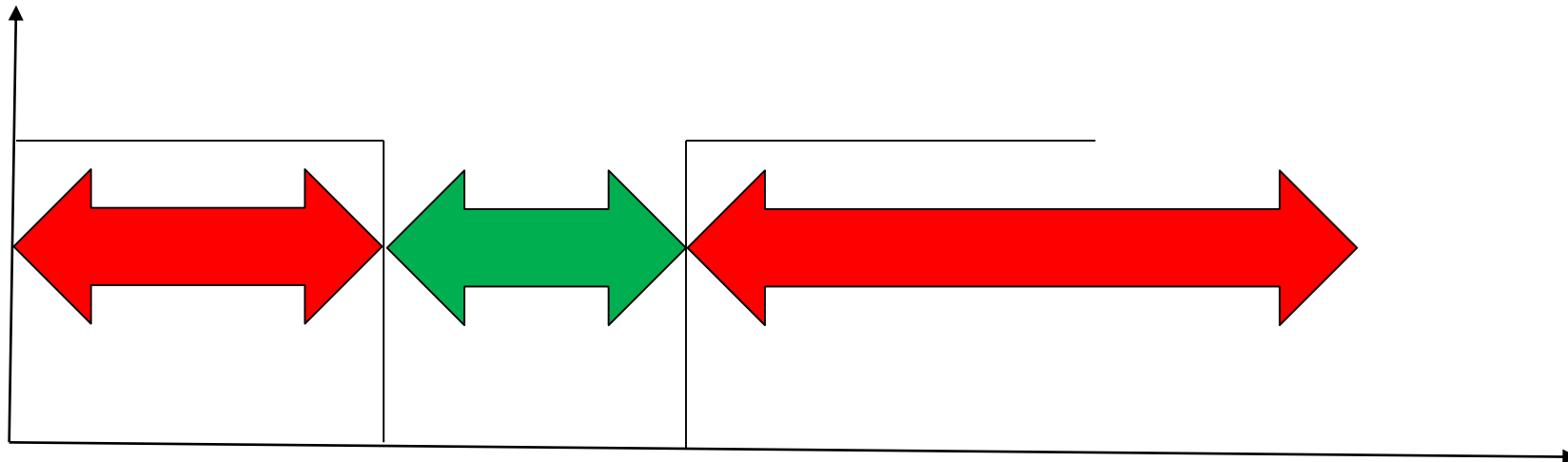


Window Watchdog

- The watchdog must be triggered within a certain window.
- I.e. also a too early trigger will cause a watchdog error

- Also validates the system clock
- Solves Freeze and Idle run of the MCU

- More complex synchronisation required (clock drift)
- No logic supervision

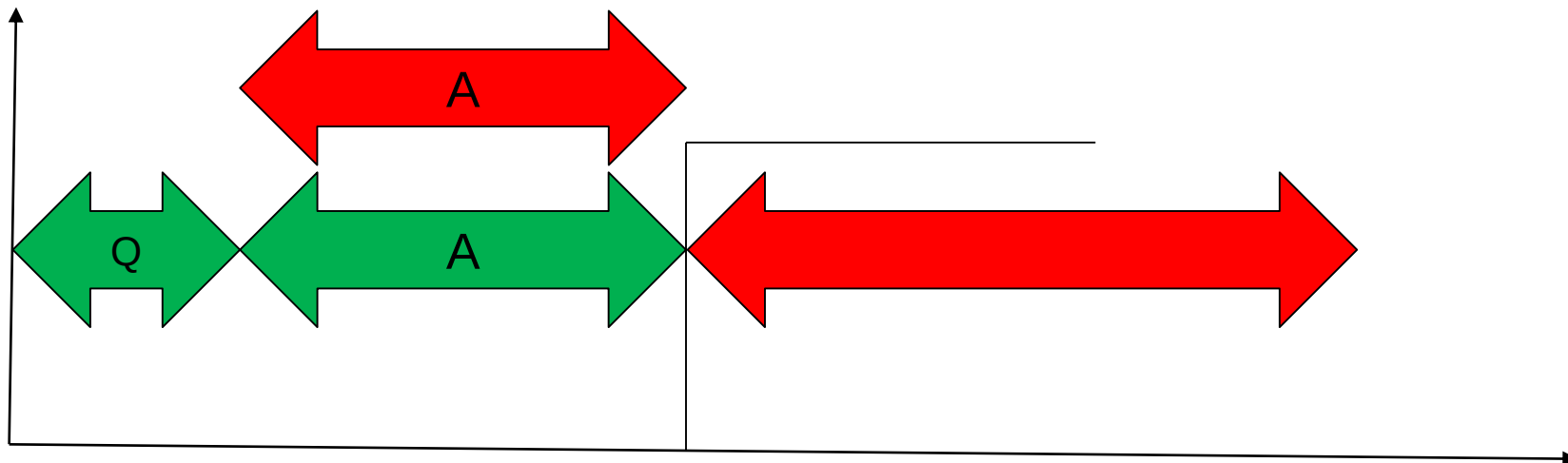


Challenge/Response Watchdog

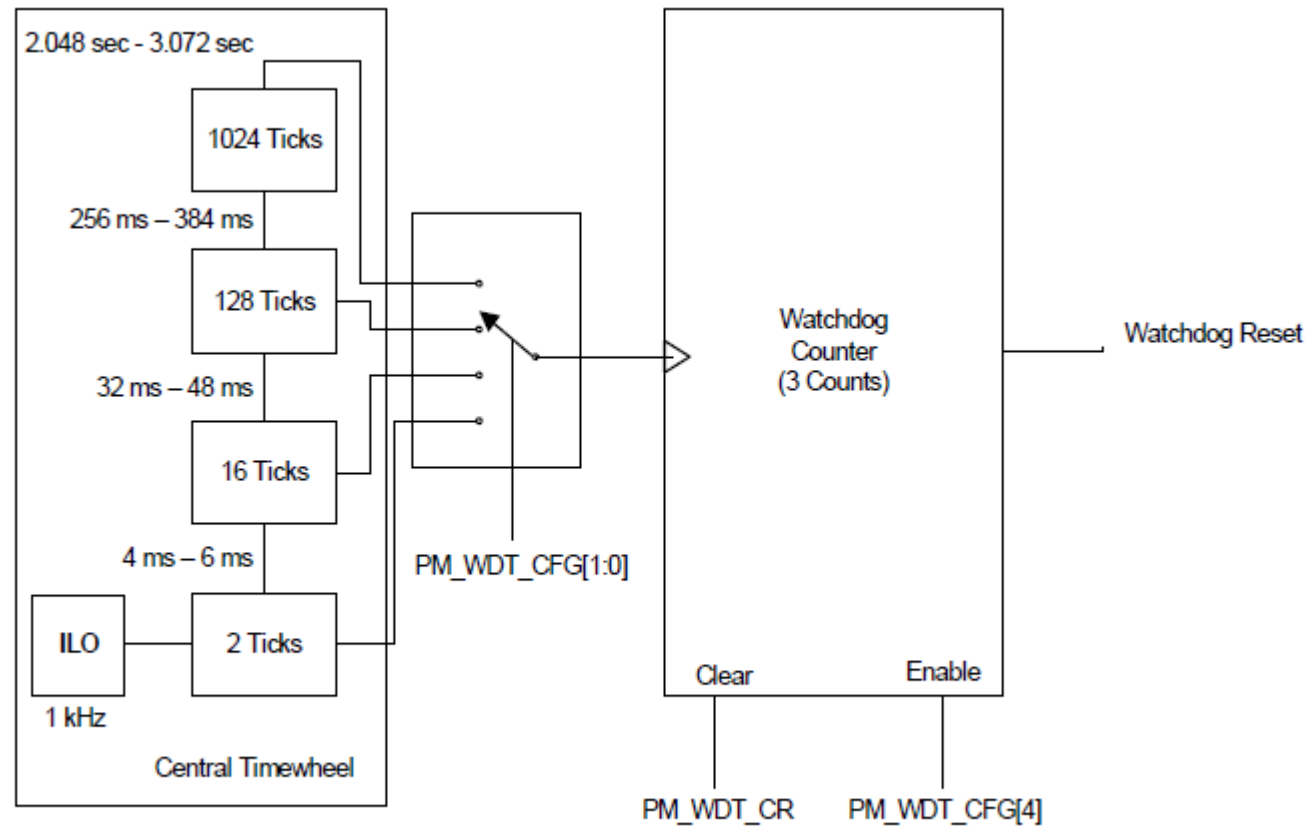
- Within a certain timeframe, the question of the watchdog must be answered correctly

- Checks the logic of the MCU
- Solves „Freeze“
- Hard to manipulate

- Complex implementation (I2C, or SPI)



PSOC Alive Watchdog



Cypress TRM

PSOC Internal Watchdog

```
//Wait, read and clear the event
WaitEvent(RTE_EVENTMASK);
GetEvent(tsk_EventDispatcher, &ev);
ClearEvent(ev);

//Process all events from the mask
RTE_ProcessEventTable(RTE_eventActivationTable, RTE_eventActivation_size, ev);

//Trigger watchdog bit for event runnables
WD_hitCheckpoint(0);

}

//Will be called every 10ms
TASK(tsk_CyclicDispatcher)
{
    //Increment the age of all signals by 1ms
    RTE_timertick();

    //Process all cyclic runnables which are due
    RTE_ProcessCyclicTable(RTE_cyclicActivationTable, RTE_cyclicActivation_size, 10);

    //Trigger watchdog bit for cyclic runnables
    WD_hitCheckpoint(1);

}

TASK(tsk_Background)
{
    while(1)
    {
        //do something with low priority

        //Brute Force Watchdog Trigger
        #if WDG == WDG_INTERN
            if (TRUE == WD_allCheckpointsHit())
            {
                WD_Trigger();
            }
        #endif
    }
}
```

If both tasks report ok, the hardware watchdog will be triggered.

Observed behaviour

After the watchdog timeout, the engine stops (at least sometimes)...

- Driver init stop the engine
- Boot Loop does not do any harm



Improved Design – check for boot reason

```

    //Check for Watchdog reset
} #if WDG_BOOTCHECK == ACTIVE
    if (TRUE == WD_CheckResetBit())
    {
        ERROR_centralHandler(ERROR_SYSTEM);
        //To avoid boot loop
        while(1)
        {
            WD_Trigger();
        }
    }
- #endif
```

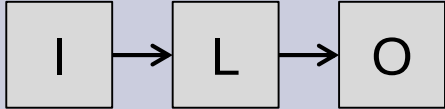
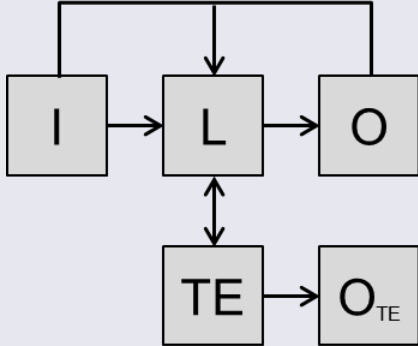
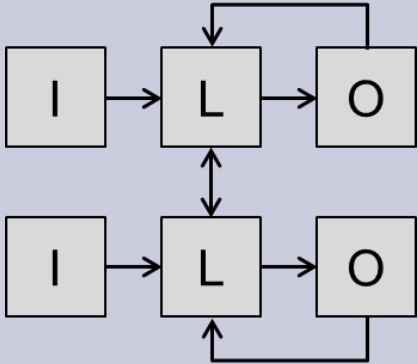

Limitations of the internal watchdog

- We assume, that at least the startup code can be executed, setting the peripherals into a safe state
- A reliable detection of the boot reason (our implementation is not really reliable) may allow a special handling, but again – is the system long enough in a safe state to run the required code?

Low power (battery) or other hardware faults may prevent a reliable transition into a safe state!

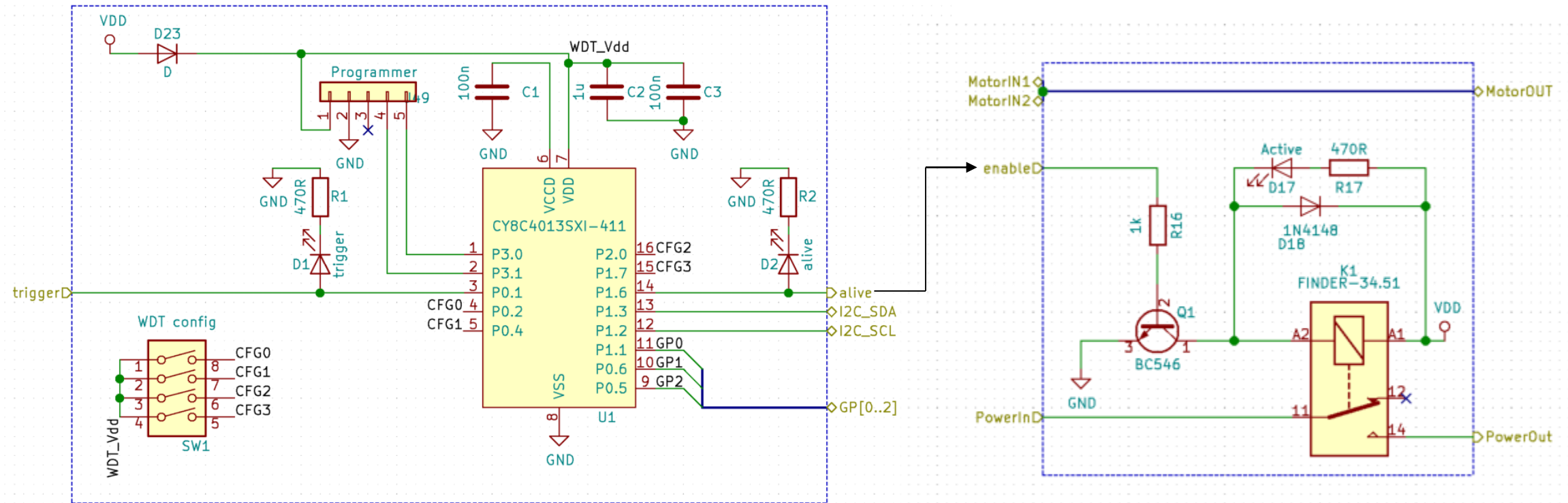
Remember this picture?

Architecture Evaluation according ISO13849

Category	Structure	Description
B / 1		• Single Channel • No Test Function • Proven in Use Hardware
2		• Single Channel • External Test Function • Safe State reachable through Test Function • Individual path are secured against common cause failures
3 / 4		• Redundant channel • Crosswise supervision • Individual path are secured against common cause failures

Improved reliability

External Watchdog Chip and Power Off



External Watchdog Chip

Functionality (controlled over DIP – on/off per feature)

- Timeout Watchdog
- Window Watchdog
- Q&A Watchdog

Trigger Input

- Rising Edge for Timeout and Window

I2C

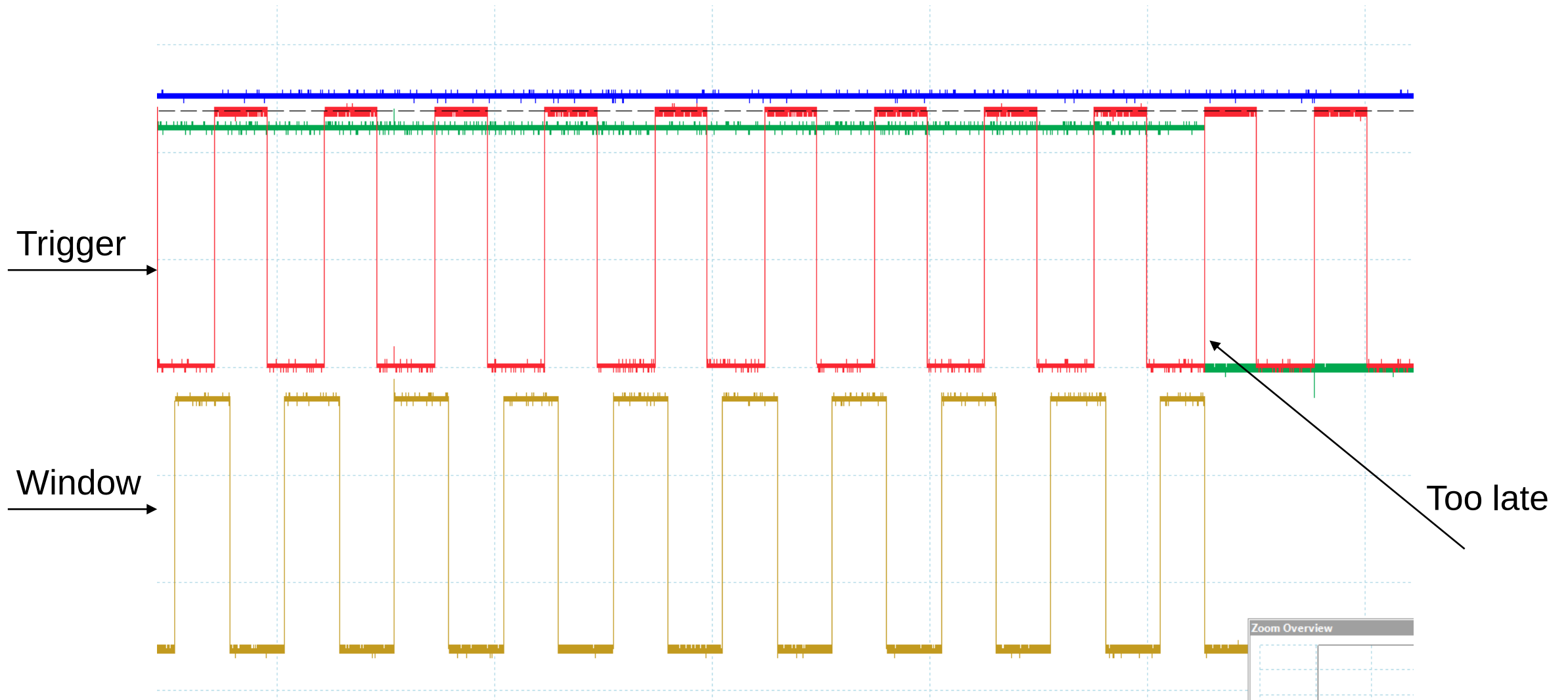
- Trigger for Q&A
- Timeout and Window Timing configuration
- No of warning IRQ before failure
- Restart after failure

GPIO

GP0: Timer Reset (IN)

- GP1: Warning / IRQ for lost trigger (OUT)
- GP2: Reset for MCU 1/2 (OUT)

Window Watchdog - Problem Timer Drift



Combining external and internal Watchdog

- At the moment, the external watchdog serves as a one shot power off.
- Once this watchdog has been hit, we require an active reset signal to be sent to the external watchdog

If we simply activate the internal watchdog in addition, the internal watchdog would reset the MCU and as a consequence the MCU would reset the external watchdog – probably without solving the problem root cause. Not what we really want.

Proper error escalation strategies and reset behavior must be defined – a non-trivial task!

How about the second problem?



Some bad user
code!!!

```
void HMI_injectError_run()
{
    //Read DIP register
    DIP_data_t dip = DIP_INIT_DATA;
    RTE_DIP_pullPort(&SO_DIP_signal);
    dip = RTE_DIP_get(&SO_DIP_signal);

    //LOG_I("HMI_injectError_run", "DIP = %x", dip.byte)

    if (1 == dip.bit.d0)
    {
        //Error Injection shutdown
        LOG_E("HMI_injectError_run", "Shutting system down");
        ShutdownOS(E_OK);
    }

    if (1 == dip.bit.d1)
    {
        //Error Injection slow down system
        LOG_E("HMI_injectError_run", "Corrupted time base");
        CancelAlarm(alrm_10ms);
        SetRelAlarm(alrm_10ms, 100, 100);
    }

    if (1 == dip.bit.d2)
    {
        //Error Injection slow down system
        LOG_E("HMI_injectError_run", "while(1)");
        while(1){;}
    }
}
```

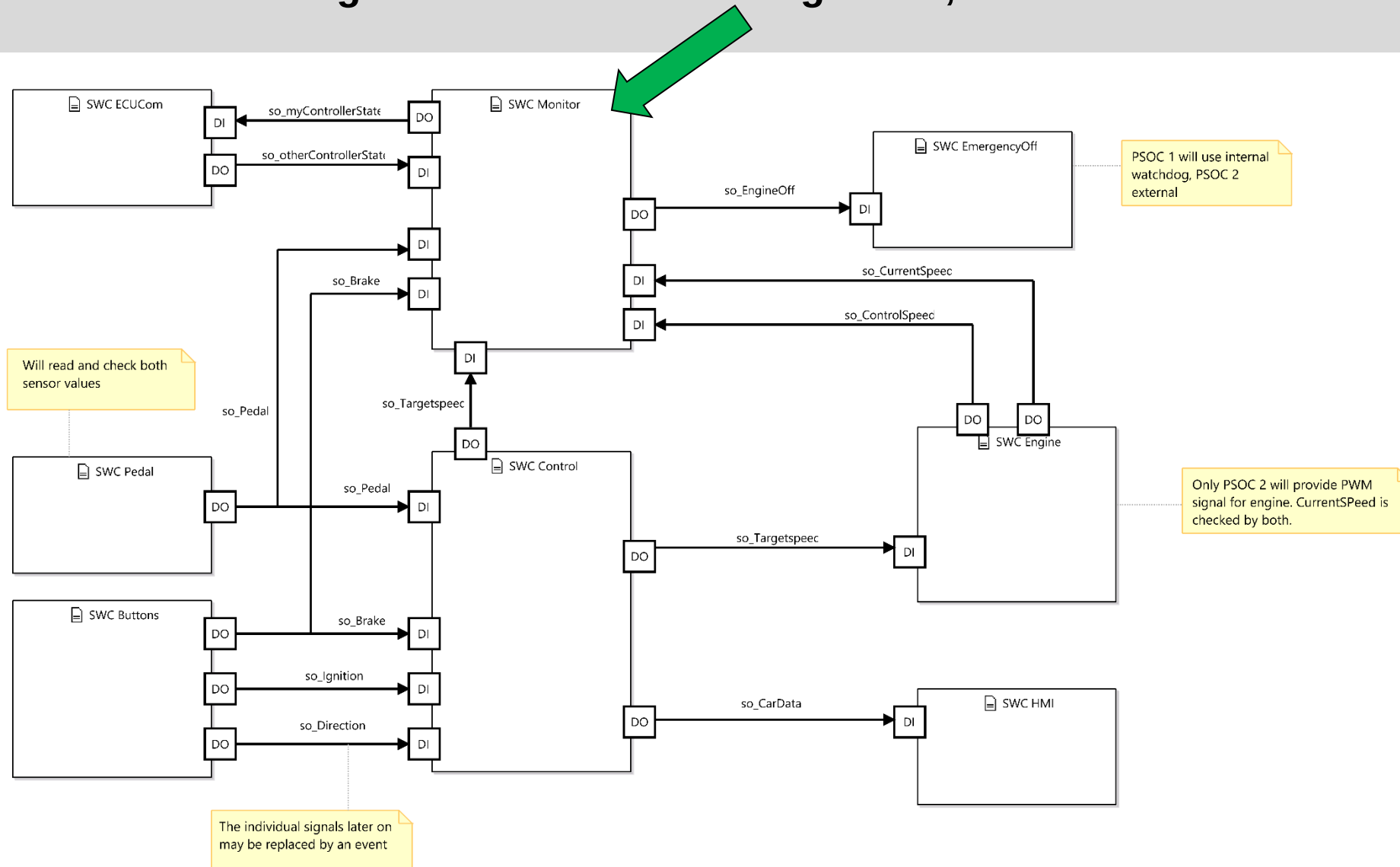
A more complex WDG
trigger bit activation in
combination with a window
watchdog might help...

Watchdog is
still triggered!

Qualitative System Verification

Id	System Requirements	ASIL _{req}	HW / SW Requirement	Status
SR5	The engine currentspeed must be in line with the controlspeed value	C	• HW: Engine power must be deactivated in case of critical failure • HW: Engine power must be controlled by independent hardware • HW: Engine current must be supervised and limited • SW: Engine Blocking must be detected • SW: Current speed must be compared with target speed • SW: Limit reverse speed • SW: Engine must be explicitly activated	
SR6	The system status must be shown to the driver	A	• SW: Provide status / error information to driver • SW: Detect freeze of OLED display	
SR7	The system integrity must be supervised	C	• HW: Provide an external watchdog, which turns of the actuator in case of a critical error • HW: Power supply must be supervised, bursts must be avoided • HW: The integrity of the controller (MCU) must be supervised • SW: The integrity of the OS must be supervised • SW: ISR bursts must be detected • SW: Low Power must be detected	

More advanced monitoring – we need a second logic unit, i.e. second controller!



Wrap-Up

- We have added an output monitor
 - More complex than input supervision, as error states are not that easy detectable
- Error handling so far required the correct execution of all runnables
 - Errors like OS Shutdown, endless loops of manipulated time bases cased dangerous malfunction
 - Concept of system state supervision using a watchdog
- Logical errors and time domain errors can still not be detected in a reliable manner
 - We have to get the second PSOC up and running